Project Interim Report

On

Bank Account Management System (Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Abhishek Kumar

RU-25-10053

Bachelor of Technology in Computer Science & Engineering in association with Google

Under the Guidance of

Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

This project is a Bank Account Management System built using core Object-Oriented Programming principles. Ensuring secure financial transactions is a critical necessity in the modern digital banking landscape. Our application tackles this by providing a reliable console-based environment where user funds are strictly controlled and manipulated through secure methodologies. The system securely manages user account balances using encapsulation, preventing unauthorized access. It allows users to safely perform standard banking operations such as deposits and withdrawals, while enforcing strict validation rules to prevent invalid actions like negative deposits or overdrafts. By leveraging data hiding and secure method design, this project successfully bridges the gap between theoretical OOP concepts and real-world software implementations.

Introduction

The Bank Account Management System is designed to simulate a real-world financial environment. It utilizes encapsulation to protect user data, ensuring that account balances are only modified through validated channels. Features include instant balance retrieval, secure deposit handling with positive-value validation, and withdrawal processing with insufficient-funds checking.

Objectives

- Manage account records securely.
- Track transactions like deposits and withdrawals.
- Reduce manual work and potential human errors in transaction handling.
- Apply OOP concepts such as encapsulation, data hiding, and method design.

Scope

This project is suitable for small banking simulations and can be extended with database integration in the future.

System Requirements

- **Hardware:** Computer with 4GB RAM

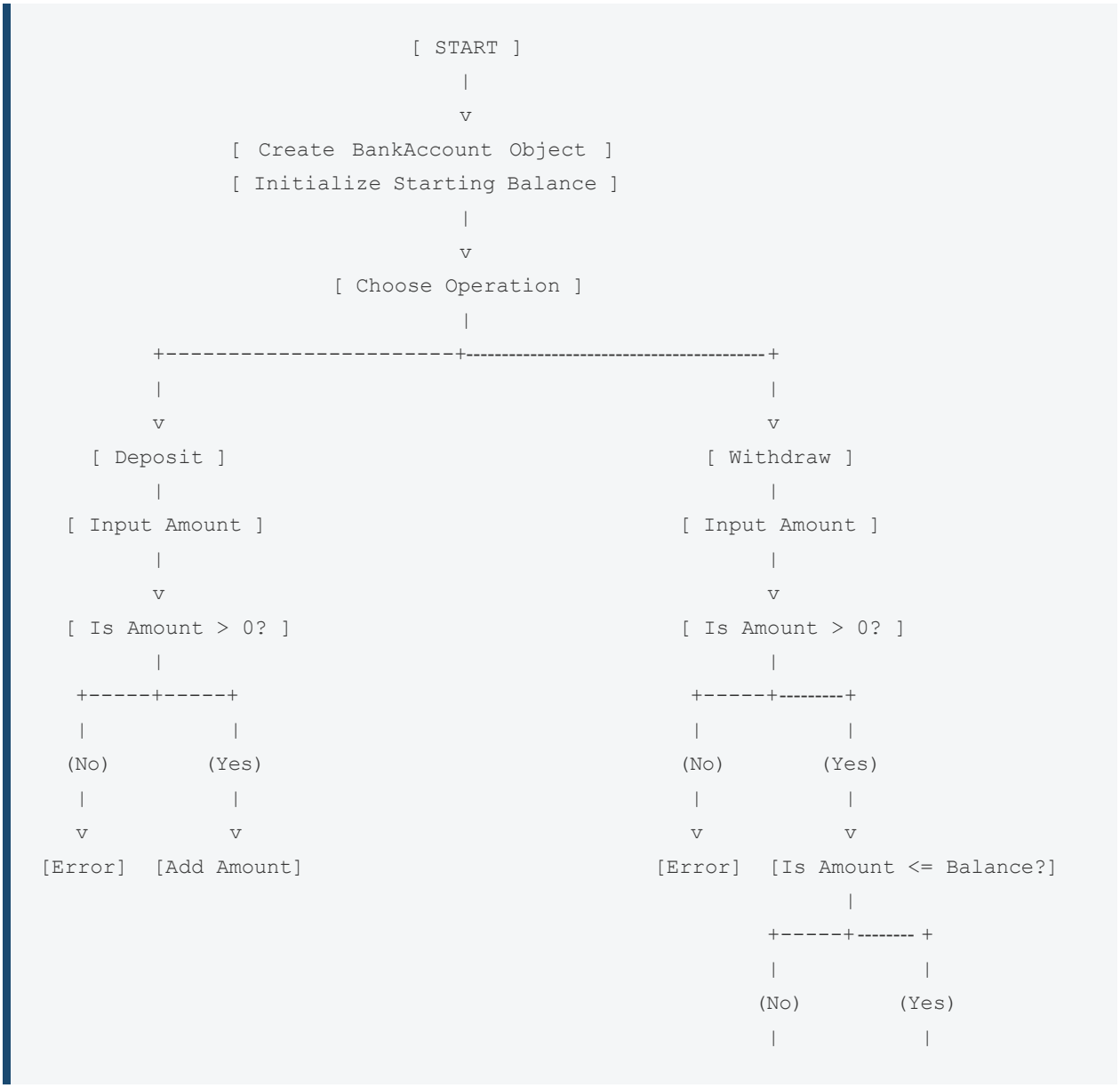
- **Software:** Java JDK, IDE (VS Code/Eclipse/Edit Plus)

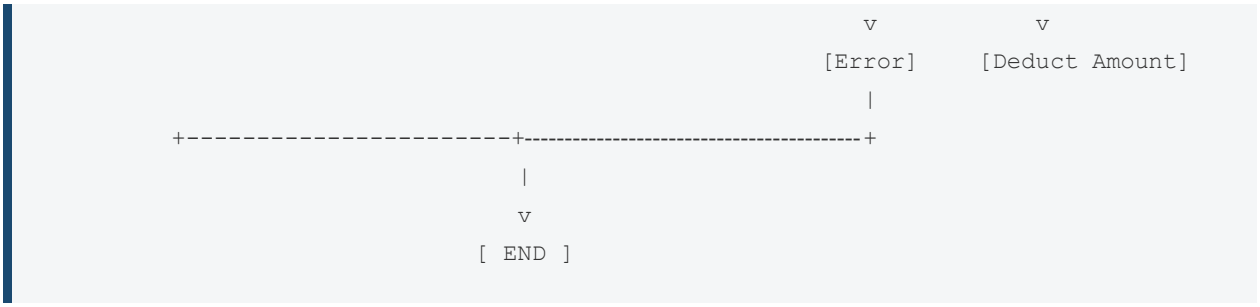
Methodology

The application flow begins by initializing a new account with a starting balance. The system then allows the user to choose an operation. For a deposit, the system validates that the input amount is positive before adding it to the total. For a withdrawal, the system checks both that the amount is positive and that it does not exceed the current available balance before deducting the funds.

Flowchart & Class Diagram

System Flowchart





Sample Class Diagram

- **ClassName:** BankAccount
- **Variables inside class:** `private double balance`
- **Methods inside class:** `public void deposit(double amount),public void withdraw(double amount),public double getBalance()`

Implementation

The core logic resides within the `BankAccount` class, which acts as the primary blueprint for user accounts. The balance variable is kept strictly private to enforce data hiding. The `deposit` method contains logical checks to reject amounts less than or equal to zero. The `withdraw` method contains two-tier logic: it first checks for negative values, and then compares the requested amount against the current balance to prevent overdrafts. A separate `Main` class serves as the driver, initializing objects and simulating user input scenarios.

Sample Input/Output

```
Initial Balance: 1000.0
Deposited: 500.0
Withdrawn: 300.0
Final Balance: 1200.0

Withdrawal failed: Insufficient balance
```

Future Enhancements

- Integration with a relational database (e.g., MySQL) to persist account data across sessions.
- Implementation of a GUI using JavaFX or Swing.
- Adding multi-user support with unique account numbers and PIN authentication.

Gantt Chart

1. Planning & Structure

A Gantt chart helps you break the project into tasks (design, coding, testing, etc.) and set timelines. Even if it's already built once, rebuilding or modifying still needs planning.

2. Time Management

It shows:

- When each task starts

- When it should finish

This helps avoid delays and last-minute panic.

Conclusion

The project successfully demonstrates Java OOP concepts and provides a functional system.

References

[1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

[2] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com.